

Product API – Hw5

This project implements the Product API based on the provided OpenAPI specification. The server is built using Go and stores product data in memory using a hashmap.

How to Run Locally

Run with Go

```
go run main.go
```

Server runs on:

```
http://localhost:8080
```

Run with Docker

Build image:

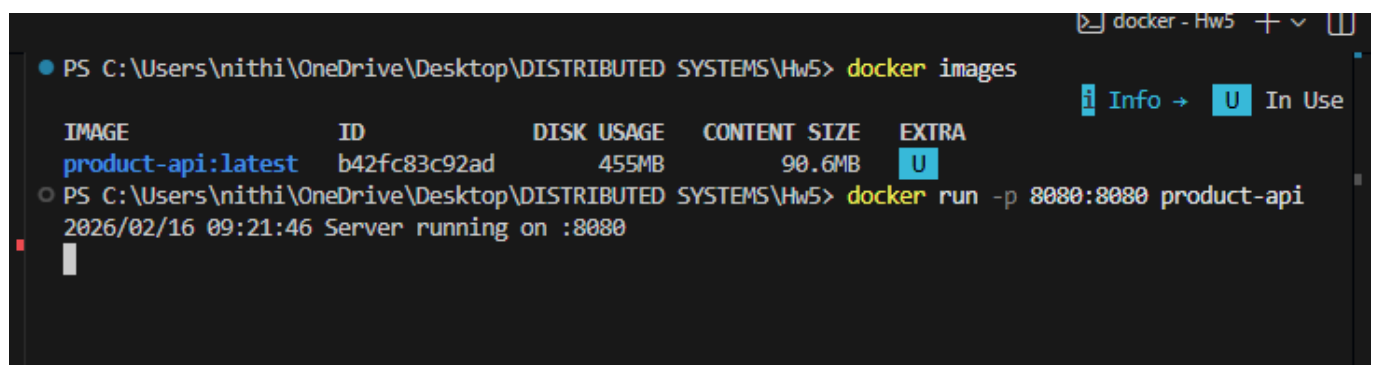
```
docker build -t product-api .
```

Run container:

```
docker run -p 8080:8080 product-api
```

Server will be available at:

```
http://localhost:8080
```



```
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw5> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
product-api:latest	b42fc83c92ad	455MB	90.6MB	U

```
PS C:\Users\nithi\OneDrive\Desktop\DISTRIBUTED SYSTEMS\Hw5> docker run -p 8080:8080 product-api
2026/02/16 09:21:46 Server running on :8080
```

API Endpoints

Create Product

POST /products

Creates a new product.

Request Body (JSON):

```
{
  "product_id": 1,
  "sku": "ABC-123-XYZ",
  "manufacturer": "Acme Corporation",
  "category_id": 456,
  "weight": 1250,
  "some_other_id": 789
}
```

Success Response:

Status: 201 Created

```
{
  "product_id": 1,
  "sku": "ABC-123-XYZ",
  "manufacturer": "Acme Corporation",
  "category_id": 456,
  "weight": 1250,
  "some_other_id": 789
}
```

HTTP

Product API Local Testing / Create Product

Save

POST

http://localhost:8080/products

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

Sc

1

{

2

"product_id": 1,

3

"sku": "ABC-123-XYZ",

4

"manufacturer": "Acme Corporation",

5

"category_id": 456,

6

"weight": 1250,

7

"some_other_id": 789

8

}

Body

Cookies

Headers (3)

Test Results

201 Created

4 ms

237 B

Sav

{}

JSON

Preview

Visualize

1

{

2

"product_id": 1,

3

"sku": "ABC-123-XYZ",

4

"manufacturer": "Acme Corporation",

5

"category_id": 456,

6

"weight": 1250,

7

"some_other_id": 789

8

}

Possible Errors:

- 400 Bad Request - Invalid input
- 409 Conflict - Product already exists

Get Product by ID

GET /products/{productId}

Example:

GET /products/1

Success Response:

Status: 200 OK

```
{
  "product_id": 1,
```

```
"sku": "ABC-123-XYZ",
"manufacturer": "Acme Corporation",
"category_id": 456,
"weight": 1250,
"some_other_id": 789
}
```

GET

http://localhost:8080/products/1

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (3)

Test Results

200 OK

2 ms

232 B

Save

JSON

Preview

Visualize

```
1 {
2   "product_id": 1,
3   "sku": "ABC-123-XYZ",
4   "manufacturer": "Acme Corporation",
5   "category_id": 456,
6   "weight": 1250,
7   "some_other_id": 789
8 }
```

Possible Errors:

- 400 Bad Request - Invalid ID
- 404 Not Found - Product does not exist

Update Product Details

POST /products/{productId}/details

Example:

POST /products/1/details

Request Body:

```
{  
  "product_id": 1,  
  "sku": "ABC-123-UPDATED",  
  "manufacturer": "Acme Corporation",  
  "category_id": 456,  
  "weight": 1500,  
  "some_other_id": 789  
}
```

POST

http://localhost:8080/products/1/details

Docs

Params

Authorization

Headers (8)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

3

4

5

6

7

8

```
"sku": "ABC-123-UPDATED",
"manufacturer": "Acme Corporation",
"category_id": 456,
"weight": 1500,
"some_other_id": 789
}
```

Body

Cookies

Headers (1)

Test Results

204 No Content

4 ms

64 B

Sav

Raw

Preview

Visualize

1

GET

http://localhost:8080/products/1

Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (3)

Test Results

200 OK

3 ms

236 B

Sav

{}

JSON

Preview

Visualize

1

2

3

4

5

6

7

8

```
{
  "product_id": 1,
  "sku": "ABC-123-UPDATED",
  "manufacturer": "Acme Corporation",
  "category_id": 456,
  "weight": 1500,
  "some_other_id": 789
}
```

Success Response:

Status: 204 No Content

(No body returned)

Possible Errors:

- 400 Bad Request
- 404 Not Found

Data Storage

Product data is stored **in memory** using:

```
map[int]Product
```

No database is used as per assignment requirements.

Validation

Input validation includes:

- `product_id` must be ≥ 1
- `sku` must be 1–100 characters
- `manufacturer` must be 1–200 characters
- `category_id` must be ≥ 1
- `weight` must be ≥ 0
- `some_other_id` must be ≥ 1
- Path ID must match body `product_id`

Error Response Format:

```
{
  "error": "ERROR_CODE",
  "message": "Description",
  "details": "Optional details"
}
```

GET

▼

http://localhost:8080/products/1

☰ Docs

Params

Authorization

Headers (6)

Body

Scripts

Settings

Query Params

	Key	Value	Description
	Key	Value	Description

Body

Cookies

Headers (3)

Test Results

↺

404 Not Found

• 4 ms

• 205 B

🌐

📄 Save

{ } JSON ▼

▶ Preview

🔍 Debug with AI ▼

☰

☰

```
1 {
2   "error": "NOT_FOUND",
3   "message": "Product not found",
4   "details": "Product ID does not exist"
5 }
```

Notes

- This is a homework assignment (Hw5) implementing an in-memory product management system
- The API follows RESTful conventions
- All data is volatile and will be lost when the server restarts